Análise de Algoritmos de Busca no Problema das Luzes

(Lights Out)

Saymon Maciel Farias
Universidade Tuiuti do Paraná
Bacharelado em Ciência da Computação
saymon.farias@aluno.utp.br

2026

Resumo

Este relatório apresenta uma análise comparativa de algoritmos de busca apli-cados ao problema das luzes (lights out). O objetivo é ilustrar, de forma didática, como estratégias de Busca em Largura, Busca em Profundidade, Busca Gulosa (Greedy), Algoritmo A* e Busca Local Simulated Annealing interagem com o problema.

1 Introdução

O puzzle Lights-out tem uma premissa simples: iniciamos com uma grade de luzes cada uma com um interruptor para ligar (1) e desligar (0) onde o objetivo principal é desligar todas as luzes, com um unico detalhe, sempre que você ligar o interruptor de uma , o estado (ligado/desligado) das 4 luzes adjacentes (acima, abaixo, esquerda e direita) também altera o seu estado.

O presente trabalho veio com o objetivo de implementar diferentes algoritmos de busca a diferentes tamanho dessas grades de luzes, para executa-los e compara-los.

2 Fundamentação Teórica

Uma definiçao precisa de problemas e as sua respectivas soluções é exencial para a implementação de algoritmos de busca de propósito geral que vão resolver esses problemas.

O processo de procurar por um conjunto de ações que alcançam um objetivo é chamado de busca. Logo, um algoritmo de busca tem como entrada um problema e devolve uma solução a partir de uma sequência de ações.

Um problema é definifo por cinco tópicos principais:
- Estado inicial: Estado em que o agente começa.
- Ações: Possíveis ações disponiveis para o agente.
- Modelo de transição: Uma descrição do que cada ação faz.
- Teste de objetivo: Determina se um estado é objetivo.
- Custo de caminho: Atribui custo numérico a cada caminho.

ESTRATÉGIAS DE BUSCA – SEM INFORMAÇÃO

BFS (Breadth-firts search) :

Em resumo o nó raiz é o primeiro a ser expandido, em seguida todos os seus sucessores são expandidos, depois os sucessores deles, e assim por diante.

A busca em largura é uma instância do algoritmo de busca em grafo, é implementado utilizando as caracteristica de uma fila (FIFO).

DFS (Depth-firt search):

A busca em profundidade, como já diz seu próprio nome, sempre vai expandir até o nó mais profundo da árvore de busca.

A busca em profundidade é uma instância do algoritmo de busca em grafo, é implementado utilizando as caracteristica de uma fila (FIFO).

ESTRATÉGIAS DE BUSCA INFORMADA – (HEURÍSTICA)

A* :

Minimização do custo total estimado da solução.

Avalia os nós através da combinação do custo do caminho e do custo para ir do nó ao objetivo.

$$f(n) = g(n) + h(n)$$

A otimalidade na busca A* necessita de que *h(n)* seja admssível, ou seja, uma heuristica que nunca supere o custo de atingir o objetivo.

GULOSA:

Conhecida como busca gulosa de melhor escolha, esse algoritmo tenta expandir o nó que está mais próximo do objetivo, tendo consciencia que isso pode direcionar a uma rápida solução.

Avalia os nós usando apenas função heurística

$$f(n) = h(n)$$

ESTRATÉGIAS DE BUSCA LOCAL

SIMULATED ANNEALING:

A ideia do algoritmo é permitir que soluções piores que a atual sejam escolhidas para exploração da vizinhança, resolvendo assim o minimo local que surge por exemplo no hill climbing.

As chances de escolher um caminho pior vai diminuindo ao longo da busca por conta do resfriamento.

```
Parâmetros:
        T0 (temperatura inicial),
        Tf (temperatura final),
        L(numero de iteracoes),
        α (redução de temperatura)

T ← T0 ; (temperatura atual recebe temperatura inicial)
S0 ← gera solução inicial; S ← S0; (solução atual)
S* ← S0; (melhor solução obtida até o momento)
```

```
enquanto T > Tf faça (temperatura alta)
      para cont ← 1 até L(T) faça (iterações para equilíbrio)
            S' ← seleciona uma solução vizinha de S
            Δcusto ← custo(S') -custo(S);
            se Δcusto < 0
                  S ← S';
            senão
                  Numero_Random ← Rand[0..1];
                  se Numero_Random < exp(-Δcusto/T)
                        S ← S';
            se (S < S*) S* ← S (guardar o melhor estado)
      fim para      T ← αT (reduz a temperatura)
fim-enquanto
Retorne S (melhor solução)
```

Figura 1- Algorimo Simulated Annealing

3 Metodologia

Para a realização da implementação dos algoritmos e validação dos resultados foi utilizado o editor de código Visual Studio Code (VS code) onde foi possivel identicar e corrigir os erros durante o desenvolvimento.

O embasamento teórico se fez a partir de referências bibliográficas que abirdavam de maneira detalhada os temas e tambem códigos/repositórios abertos do Git Hub, que serviram como base para realizar a implementação correta e eficiente dos algoritmos de busca.

4 Resultados

Algoritmo	3x3	5x5.	10x10.	20x20
BFS	0.0055s	não terminou	não terminou	não terminou
DFS	0.0030s	11.8079s	não terminou	não terminou
Gulosa	0.0030s	0.0641s	não terminou	não terminou
A*	0.0060s	não terminou	não terminou	não terminou
Simulated Annealing	0.0010s	0.0065s	0.0090s	0.0140s

Tabela 1: Tempo de execução dos algoritmos

Algoritmo	Encontrou solução	Aceitável	Ótima
BFS	Sim	Sim	Sim
DFS	Sim	Sim	Sim
Gulosa	Sim	...	...
A*	Sim	...	...
SA	Sim	Não	Não

Tabela 2: Qualidade das soluções em tabelas 4x4

5 Discussão

Ao executar os algoritmos foi estipulado um tempo limite de 5 minutos de espera cada tamanho de tabela, onde, ao estrapolar esse tempo considera-se que o mesmo não terminou a execução. Foi observado que em problemas onde a tabela ultrapassa o limite 4x4 quase todos os algoritmos de busca demandam de muito tempo de processamento para achar uma solução devido ao crescimento exponecial de de estados, característica comum em buca não-infromada

No A* eu foi abordado heuristica fraca, onde só era contado o número de células afetadas o que não estrapolava o custo real, mas acabou ficando redundante, como consequência, o algoritmo apresentou desempenho inferior ao esperado.

O Simulated Annealing demonstrou um ótimo desempenho, omde resolveu todas as instâncias testadas em tempos significativamente reduzidos. Esse comportamento reforça a vantagem de algoritmos de busca local em problemas com grandes espaços de estados.

6 Conclusão

Este trabalho teve como objetivo analisar o desempenho de diferentes algoritmos de busca aplicados ao problema Lights Out, considerando diferentes tamanhos de tabuleiro. A partir dos experimentos realizados, foi possível observar que algoritmos de busca clássicos, como BFS e DFS tem dificuldade no tratamento de grande volume de dados e de escalabilidade, se apresemtamdo inviáveis para esses casos.

Por outro lado, algoritmos de busca local, como o Simulated Annealing, demonstraram desempenho superior, sendo capazes de resolver todas as instâncias testadas em tempo reduzido.

Dessa forma, é visto que para problemas com grande escalabilidade, resoluções baseadas em busca local são mais adequadas do que métodos tradicionais.

Referências

1- https://medium.com/random-noise/solving-medium-sized-lights-out-puzzles-with-pythonbaa41890edfa

2- S. J. Russell & P. Norvig. Artificial Intelligence: A Modern Approach. Prentice Hall, 3rd edition, 2010.

3- https://www.inf.ufpr.br/aurora/disciplinas/topicosia2/downloads/trabalhos/SimulatedAnnealingTSP.pdf